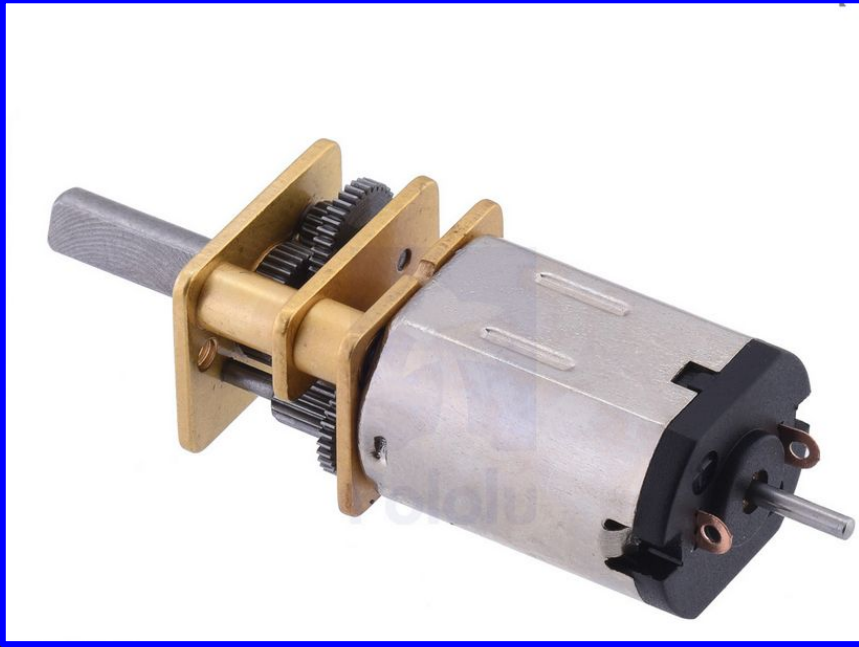


Motors and Encoders

IEEE @ UCI

What is a DC (direct current) motor?



- A machine that converts electrical energy to mechanical rotation
- We use the motor to spin a wheel

What DC motor to we use?



- Pololu Micro Metal Gearmotor
- Dual shaft
- 50:1 gear ratio
- We use the motor to spin a wheel
- We use 2 motors to spin 2 wheels



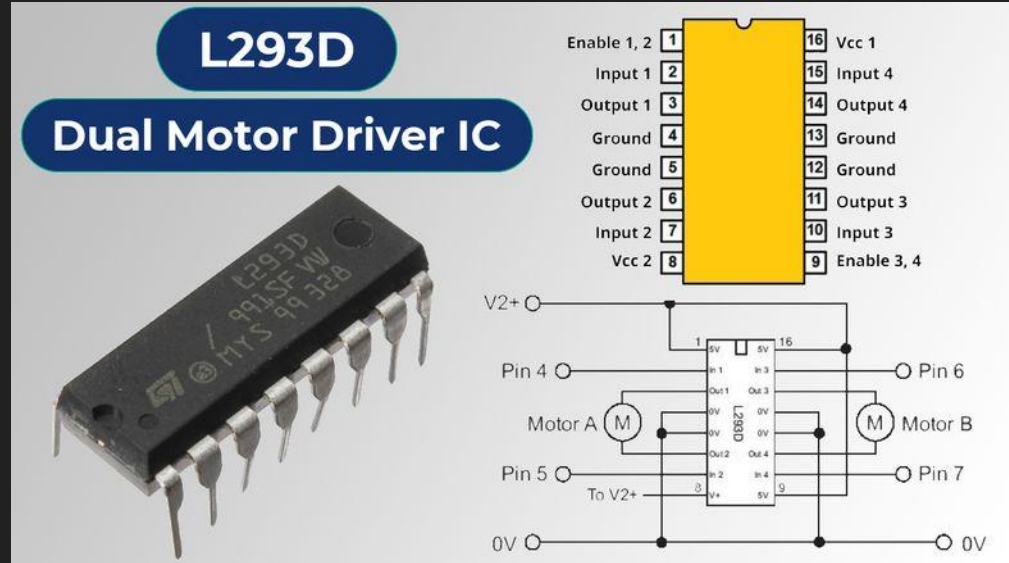
How do we install the motors?

- Motors are held in place by motor mount screwed into PCB



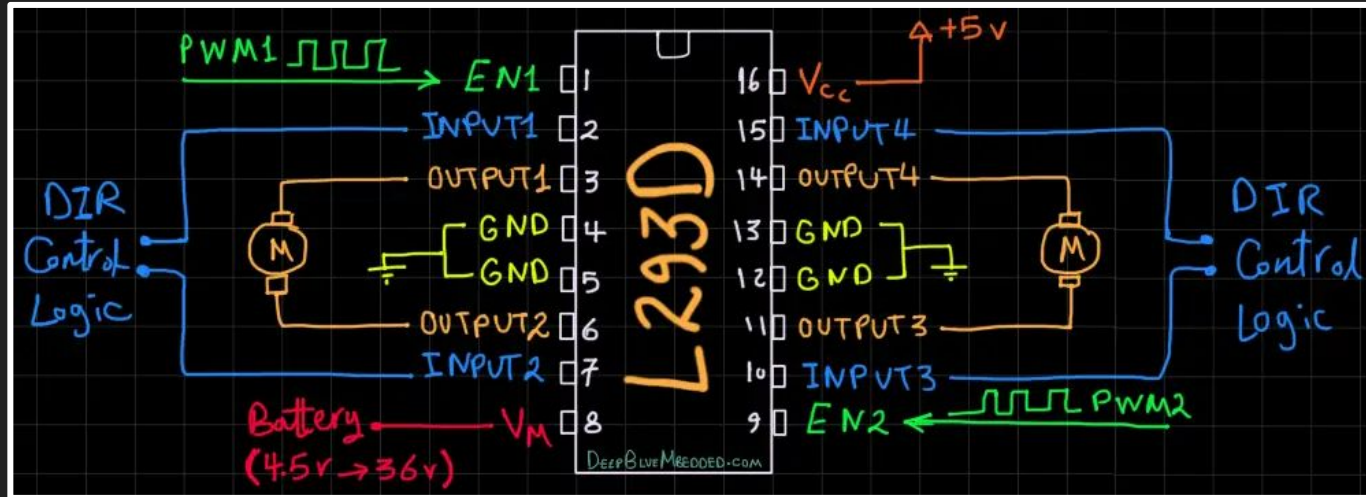
How do we spin a motor?

- We use a motor driver: L293D
- A motor driver is an IC (integrated circuit) chip that controls how the motor spins
- The input to control the spin is a PWM signal



How do we spin a motor?

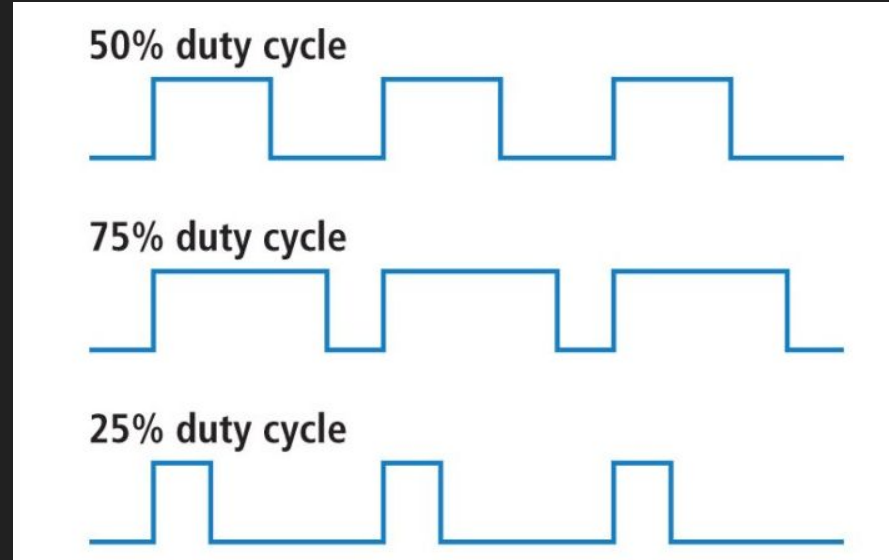
- PWM - Input signal to set motor speed
- Logic Voltage Input - Used to power the motors



- Battery Input - Used to power the motors
- Motor Direction Input - Used to set clockwise or counterclockwise rotation

What is PWM?

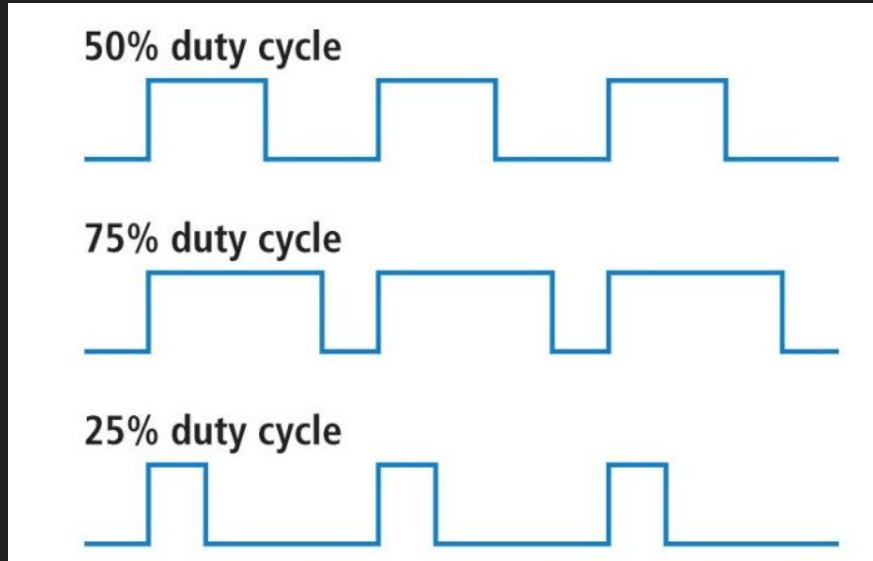
- PWM → Pulse Width Modulation
- A signal that is high for $x\%$ of the period and low for $(1-x)\%$ of the rest of the period
- This $x\%$ is called the duty cycle percentage
- You will set a value from 0 to 2^X , where X is the PWM bit resolution



What is PWM?

- The PWM signal can simulate a voltage
- The simulated voltage is equal to the amplitude of the signal * duty cycle

Amplitude:
10V



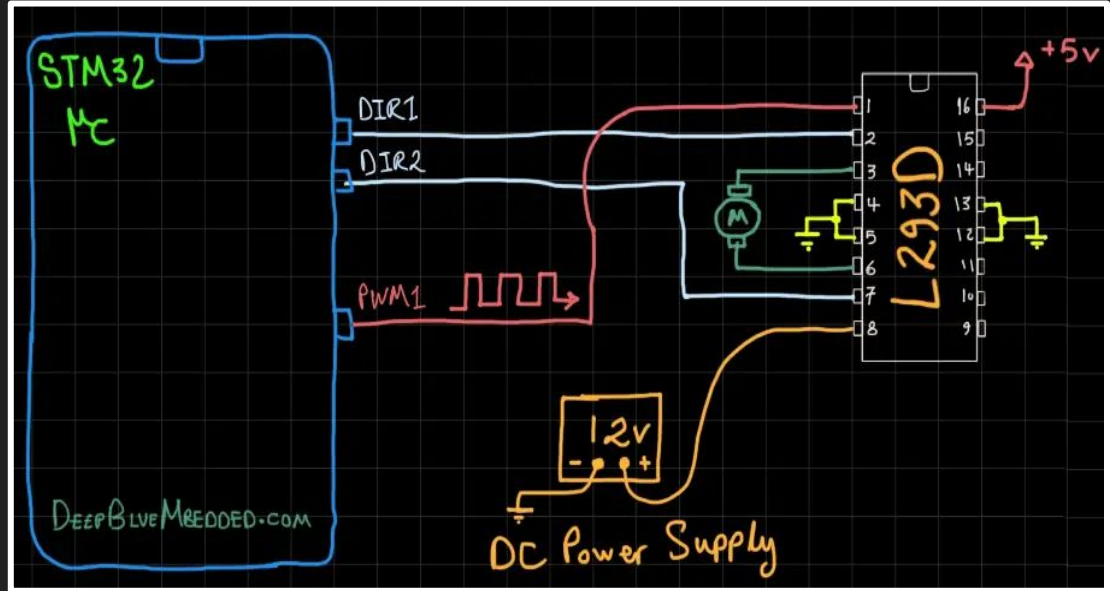
$$= 10V * 50\% = 5V$$

$$= 10V * 75\% = 7.5V$$

$$= 10V * 25\% = 2.5V$$

Motor Driver in Depth

- Set DIR outputs
- Set PWM signal duty cycle
- L293D uses input duty cycle, to output PWM signal of with equal duty cycle %, but the amplitude of power supply voltage



Motor Driver in Depth

- How do we define and set the DIR signal?
 - Define two **GPIO** pins as OUTPUT per motor
 - 1 GPIO pin is CW (clockwise)
 - 1 GPIO pin is CCW (counter clockwise)
- How do we define and set the PWM signal?
 - We use a **PWM** output pin on the STM32

Direction signals



- ```
86 #define MR_FWD_Pin GPIO_PIN_13
87 #define MR_FWD_GPIO_Port GPIOB
88 #define ML_BWD_Pin GPIO_PIN_14
89 #define ML_BWD_GPIO_Port GPIOB
90 #define MR_BWD_Pin GPIO_PIN_15
91 #define MR_BWD_GPIO_Port GPIOB
92 #define ML_FWD_Pin GPIO_PIN_8
93 #define ML_FWD_GPIO_Port GPIOA
```

- $\text{GPIOA} \rightarrow \text{PORTA} \rightarrow \text{PA}_x$
- $\text{GPIOB} \rightarrow \text{PORTB} \rightarrow \text{PB}_x$

# STM32 Setup - Direction

## Example code to set direction

```
86 #define MR_FWD_Pin GPIO_PIN_13
87 #define MR_FWD_GPIO_Port GPIOB
88 #define ML_BWD_Pin GPIO_PIN_14
89 #define ML_BWD_GPIO_Port GPIOB
90 #define MR_BWD_Pin GPIO_PIN_15
91 #define MR_BWD_GPIO_Port GPIOB
92 #define ML_FWD_Pin GPIO_PIN_8
93 #define ML_FWD_GPIO_Port GPIOA
```

- For Left Motor, setting direction as FWD

```
HAL_GPIO_WritePin(GPIOA, 8, 1); // sets ML_FWD as HIGH
HAL_GPIO_WritePin(GPIOB, 14, 0); // sets ML_BWD as LOW
```

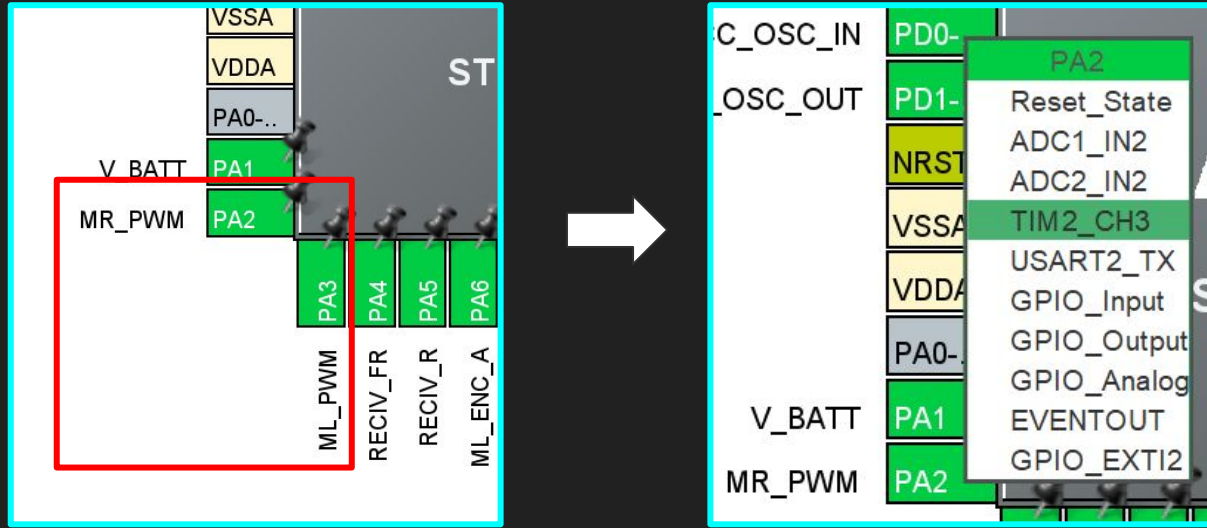
- For Left Motor, setting direction as BWD

```
HAL_GPIO_WritePin(GPIOA, 8, 0); // sets ML_FWD as LOW
HAL_GPIO_WritePin(GPIOB, 14, 1); // sets ML_BWD as HIGH
```

# STM32 Setup - Motor Speed

PWM Video: <https://www.youtube.com/watch?v=AjN58ceQaF4>

We use the PWM signal to drive the desired speed to the motor driver



# STM32 Setup - Motor Speed

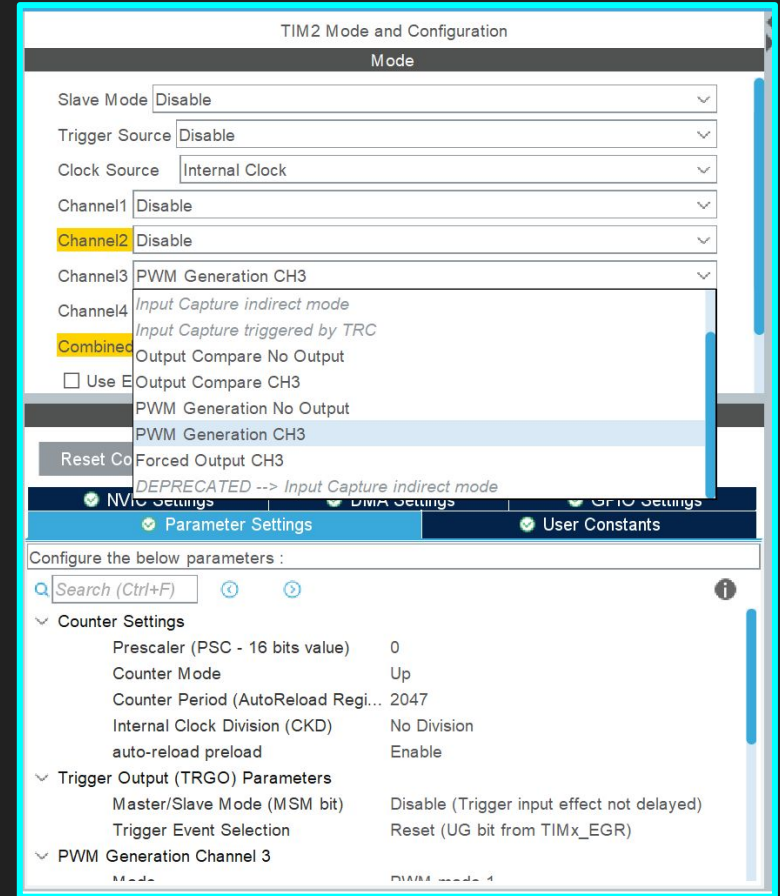
To set the parameter settings use this:

[STM32 Guide #3: PWM + Timers](#)

Make sure to use:

```
HAL_TIM_PWM_Start(htim*, pwm_channel)
```

To activate pwm output



# STM32 Setup - Motor Speed

Data Types:

```
TIM_HandleTypeDef htim2;
```

\*NOTE\* Check function:  
MX\_TIM2\_Init(void)

```
htim2.Instance = TIM2;
htim2.Init.Period = 2047;
```

To set PWM duty cycle, set manually:

For ML\_PWM → TIM\_CHANNEL\_4:

```
TIM2->CCR4 = duty_cycle_value;
```

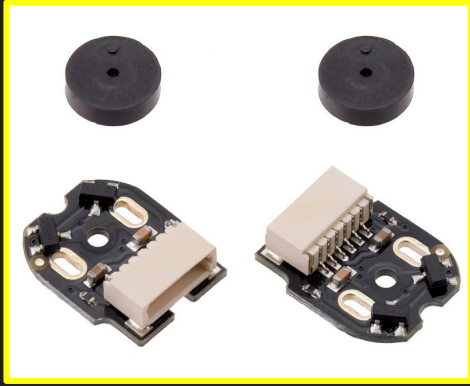
For MR\_PWM → TIM\_CHANNEL\_3:

```
TIM2->CCR3 = duty_cycle_value;
```

How do we keep track of distance traveled?



# Encoders



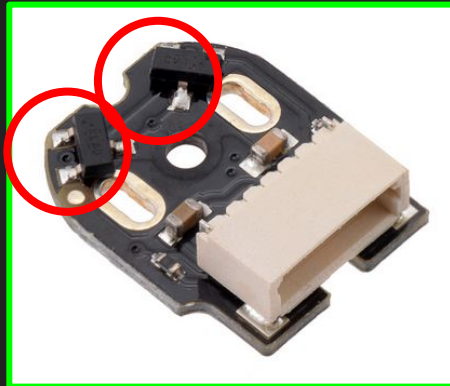
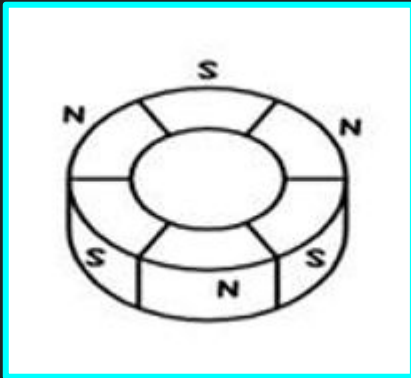
- Encoders output “ticks” as they are rotated
- Pololu encoders output 12 ticks per rotation
- Out motors have a 50:1 gear shaft ratio
- 1 rotation of the wheel shaft → 50 rotations of the encoder magnet
- $50 \times 12 = 600$  ticks per rev



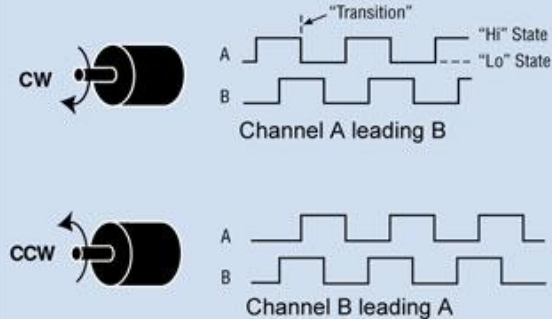
# Encoders



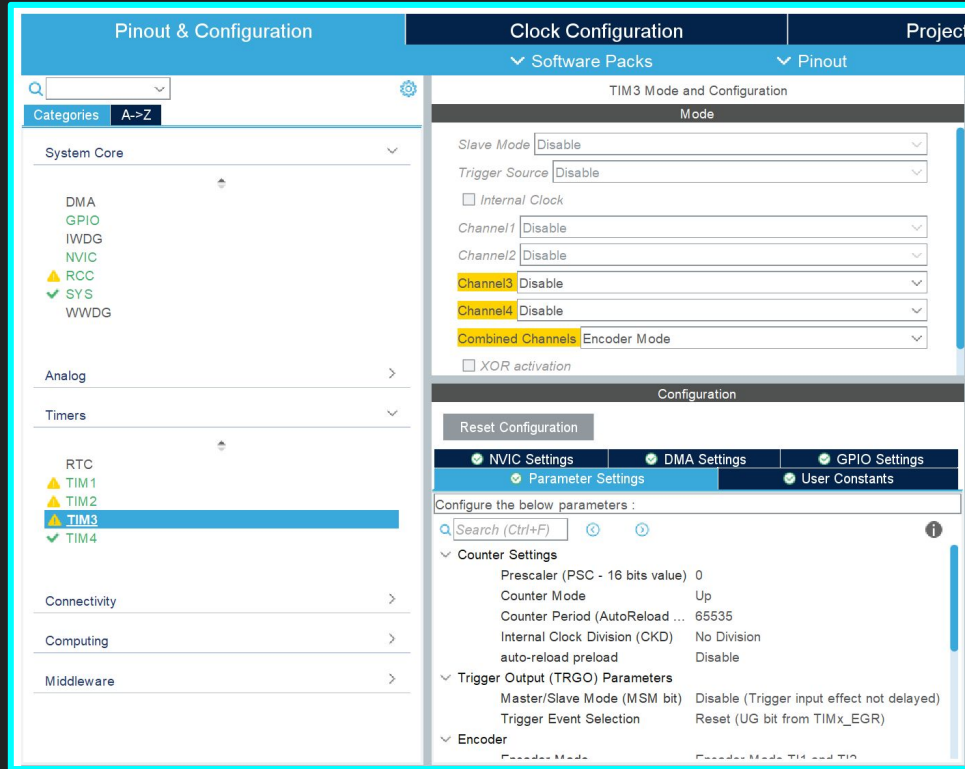
- The Pololu encoders have 6 poles
- The Hall effect sensors detect changes in poles
- Channel A and B for each sensor, output HIGH or LOW when positive/negative pole above it
- STM32 has an “Encoder Mode” setting that simplifies the software implementation to count ticks



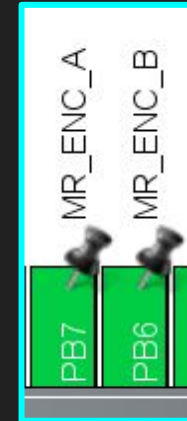
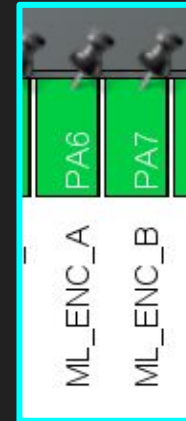
## Quadrature



# Encoders - CubeIDE



- Configure TIM3 and TIM4 as Combined Channels → Encoder Mode
- IOC pins will automatically be defined



# Encoders - CubeIDE

Configure the below parameters :

Search (Ctrl+F)

auto-reload preload Disable

Trigger Output (TRGO) Parameters

Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Reset (UG bit from TIMx\_EGR)

Encoder

Encoder Mode Encoder Mode TI1 and TI2

Parameters for C...

Polarity Falling Edge

IC Selection Direct

Prescaler Division Ratio No division

Input Filter 0

Parameters for C...

Polarity Falling Edge

IC Selection Direct

Prescaler Division Ratio No division

Input Filter 0

- Set Parameter Settings → Encoder Mode as: “Encoder Mode TI1 and TI2”
- Change Polarity to Falling Edge for both
- Go to “NVIC Settings” and enable global interrupts for both TIM3 and TIM4

|                                                                                                      |                                                                                                  |                                                                                                    |              |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|--------------|
|  NVIC Settings      |  DMA Settings |  GPIO Settings  |              |
|  Parameter Settings |                                                                                                  |  User Constants |              |
| NVIC Interrupt Table                                                                                 | Enabled                                                                                          | Preemption Priority                                                                                | Sub Priority |
| TIM3 global interrupt                                                                                |               | 0                                                                                                  | 0            |

# Encoders - Interrupts

Implementation reference video: [link](#)

```
// Very Basic Simple Encoder Interrupt Handling
```

```
uint16_t enc_left = 0;
```

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim) {
```

```
 // this is the left encoder timer
```

```
 if (htim->Instance == TIM3) {
```

```
 enc_left = __HAL_TIM_GET_COUNTER(htim);
```

```
 }
```

```
}
```

## \*NOTE on software implementation\*

- Overflow and underflow, the encoder count register that its pointed to from the `__HAL_TIM_GET_COUNTER` macro counts up to 65535 and then goes to 0,
- If the encoder spins backwards and is counting down past 0, it will underflow to 65535
- How to handle underflow / overflow?
  - Consider having a absolute encoder count variable that can be negative or positive
  - In the interrupt see how the encoder tick value has changed,  $0 \rightarrow 65535$  or  $65535 \rightarrow 0$
  - Maybe factor in motor direction
  - There are many ways to implement an “objective” tick count